

Stripe Revenue Display

Design and build specification for a 1.54" 240x240 LCD revenue instrument

Version 1.0 | August 2026

1. Concept

A single-purpose desk instrument that displays live Stripe revenue metrics. It answers one question at a time, in numbers large enough to read across a room, and talks to nothing except the Stripe API.

Design principles:

1. One question per screen. Never two.
2. Every glyph must be legible at 50cm without leaning in.
3. The device is an appliance, not a dashboard. No interaction, no menus, no touch.
4. It never lies. A stale number is visibly marked stale.
5. It is disconnected from the product it measures. No shared database, no webhooks, no backend.

2. Physical constraints

This is the section that governs every other decision.

2.1 Pixel density

A 1.54" square panel has sides of $1.54 / \sqrt{2} = 27.7\text{mm}$. At 240 pixels across:

240 px / 27.7 mm = 8.7 px/mm (approximately 220 PPI)

2.2 Legibility floor

Comfortable reading at arm's length (50cm) requires roughly 2.5 to 3mm cap height. Converting:

Pixel size	Physical height	Verdict at 50cm
11 px	1.3 mm	Unreadable
13 px	1.5 mm	Unreadable
15 px	1.7 mm	Squint
20 px	2.3 mm	Marginal
24 px	2.8 mm	Minimum viable
34 px	3.9 mm	Comfortable
60 px	6.9 mm	Glanceable
88 px	10.1 mm	Across the room

Rule: nothing below 20px. Nothing below 24px for anything a user needs to read reliably.

This is the single most common failure when designing for small panels. Web typography habits (11 to 13px labels) are calibrated for ~100 PPI monitors, where 12px is about 3mm. On this panel the same value is 1.3mm. Always convert to millimeters before approving a layout.

2.3 Text column width

With 16px padding on each side, the usable text column is **208px**.

DejaVu Sans Mono (and most monospace faces) have an advance width of approximately **0.6em**. Therefore:

```
max_chars = 208 / (font_size * 0.6)
```

Font size	Max characters
96 px	3
88 px	3
64 px	5
52 px	6
24 px	14
22 px	15
20 px	17

Consequence: the entire screen holds about 8 lines of text at the legibility floor. That is the whole content budget.

2.4 Hero sizing rule

Do not hardcode hero font sizes per screen. Compute them, so a customer at \$145k MRR does not overflow:

```
size = min(96, floor(208 / (len(value_string) * 0.6)))
size = largest_available_bitmap_size <= size
```

3. LCD-specific constraints

This panel is IPS LCD, not AMOLED. Three consequences:

3.1 No true black

The backlight is always on. #000000 renders as dark charcoal with visible edge bleed around glyphs. Designs that assume "numbers floating in void" will not land. Use #121212 rather than pure black; it is honest about what the panel can do.

3.2 RGB565 color depth

The panel accepts 16-bit color: 5 bits red, 6 bits green, 5 bits blue. Consequences:

- Subtle grays band visibly. Avoid gradients entirely.
- Blue has the fewest bits and quantizes worst.

- Verify your palette round-trips through 5-6-5 before committing. Convert each hex yourself and compare, rather than letting the driver truncate silently.

3.3 Backlight, not pixels, consumes power

Unlike AMOLED, dark backgrounds save no power. A dark theme is an aesthetic choice, not an efficiency one. Budget for the backlight running continuously.

Backlight lifetime is approximately 20,000 to 30,000 hours, or about three years of 24/7 operation. Implement a PWM dimming schedule (for example, 20% brightness between 10pm and 7am). This roughly doubles lifetime and stops the device lighting the room at night.

3.4 Viewing angle

IPS holds color well off-axis, but blue shifts most. Since a desk object is typically viewed from above at 30 to 45 degrees, avoid blue as a foreground accent.

4. Color palette

4.1 Chosen palette: neutral dark

Role	Hex	Usage
Background	#121211	Screen field
Primary text	#F4F2EC	Hero numbers, key values
Muted text	#8E8C84	Labels, subtitles, context
Dim text	#6B6A64	Footer, version, status
Inactive	#3A3A37	Unfilled rotation dots
Accent green	#5DCAA5	Realized gains only
Accent amber	#EF9F27	Degraded states only

4.2 Color discipline

- **Green means exactly one thing: realized positive movement.** Not trials, not counts, not conversion rates. If everything is green, green means nothing.
- **Amber appears only in degraded states** (stale data, auth failure).
- **Red is not in the base palette.** Add it only for threshold breaches (for example, churn above a configured limit), so its appearance carries information.

4.3 Why not Stripe's brand palette

Stripe's documented tokens are Blurple #635BFF, Dark Navy #0A2540, slate text #425466, light surface #F6F9FC, border #E6EBF1, cyan #00D4FF, success green #24B47E, error red #CD3D64.

Two reasons the project uses neutral dark instead:

Technical. Blurple has a relative luminance of about 0.17. Against a near-black background that yields roughly 4:1 contrast, below the 4.5:1 WCAG AA threshold, and it looks muddy at distance. Blue also suffers most from RGB565 quantization and the largest off-axis shift. Blurple works as a background with white text, not as a foreground accent on dark.

Commercial. If the device is sold, Stripe's exact brand colors plus their name creates an implied-endorsement problem. Nominative fair use covers "works with Stripe" in plain text; a navy-and-blurple instrument with a Stripe wordmark reads as an official product.

Strategic. A neutral palette keeps the door open for Paddle, Lemon Squeezy, or Chargebee support later. The hardware and layout are provider-agnostic; only the polling layer is Stripe-specific. A device shipped in Stripe navy is awkward to sell to a Paddle customer.

Alternative palettes evaluated and rejected for this build, retained for reference:

- **Stripe navy:** background #0A2540, text #FFFFFF, accent cyan #00D4FF. Cyan clears contrast where blurple does not.
- **Stripe light:** background #F6F9FC, text #0A2540, accent blurple #635BFF at 5.9:1. Sharpest option on IPS since it avoids backlight bleed, but requires mandatory night dimming.

5. Layout system

5.1 The three-zone skeleton

Every screen, including error states, uses identical zone positions. This is what makes rotation feel like one instrument changing state rather than four screens flashing by.

```
+-----+ y=0
| [16px padding] |
| LABEL 20px, muted | y=16
| |
| |
| |
| |
| HERO VALUE 52-96px, primary | baseline y=150
| subtitle 22px, context | baseline y=178
| |
| footer / dots 18px, dim | y=210
+-----+ y=240
```

- Label baseline: y=16, left-aligned at x=16
- Hero baseline: y=150
- Subtitle baseline: y=178
- Footer or rotation dots: y=210
- Padding: 16px all sides

5.2 Why left-aligned, not centered

Left alignment gives a stable optical anchor as values change width. Centered numbers appear to shift horizontally when 94 becomes 100, which draws the eye to a non-event.

5.3 Partial update window

Because the skeleton never moves, only the middle block changes between screens. That is a fixed rectangle of approximately **240 x 110 pixels**.

At RGB565, that is $240 \times 110 \times 2 = 52.8 \text{ KB}$. Over SPI at 40 MHz that is roughly **11ms**, imperceptible.

A full-screen redraw is $240 \times 240 \times 2 = 115 \text{ KB}$, about 25ms, which reads as a visible flicker. Use the ST7789 column and row address window commands (CASET / RASET) to push only the changed region. Full redraws are acceptable only on deliberate rotation transitions, where a wipe reads as intentional.

5.4 Typography

- **Monospace throughout.** Tabular figures prevent numbers jittering when 94 becomes 100.
- **Slight negative letter-spacing on hero values** (about -0.02em) pulls monospace digits closer to a proportional appearance at large sizes. This is the single largest visual improvement available.
- **Use bitmap fonts, not runtime-rendered TrueType.** At 220 PPI without subpixel rendering, anti-aliased proportional text looks fuzzy. You need only digits, currency symbols, and about a dozen uppercase letters. A hand-tuned glyph set at your specific sizes is roughly 20KB and looks noticeably sharper.
- **Separators:** use / or , rather than middot. A middot at 22px is about 1mm of ink and disappears.

5.5 What to avoid

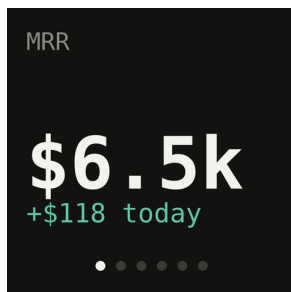
- **Divider rules.** A 2px line is 0.23mm and reads as a rendering artifact. Use whitespace and space-between instead.
- **Gradients.** Band visibly in RGB565.
- **Sparklines and progress bars.** Tested and rejected: at this size they cannot carry enough resolution to be informative, and they consume space a legible number needs.
- **Icons.** At 24px an icon is 2.8mm, ambiguous. A word is clearer.

6. Screen deck

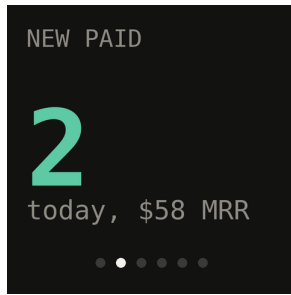
Nine screens: six rotating metrics and three device states.

6.1 Rotation screens

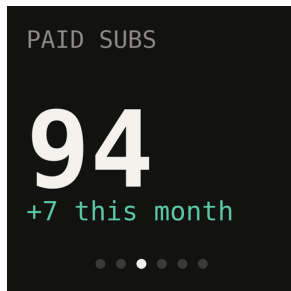
Rotate every 8 seconds. Dots at the footer show position.



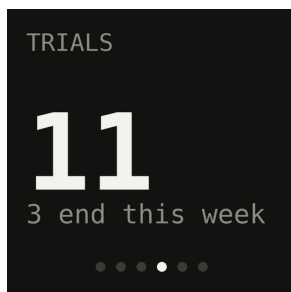
1. MRR. The anchor metric. Abbreviated to \$6.5k rather than \$6,512 because five glyphs at 60px fit the column while six do not, and precision below \$100 is not decision-relevant at a glance.



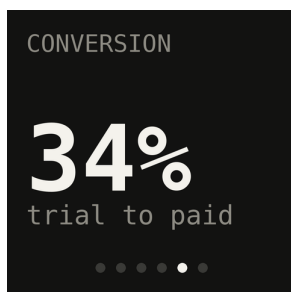
2. New paid today. The delta most worth checking repeatedly. Green because it is a realized gain. The subtitle carries the revenue impact.



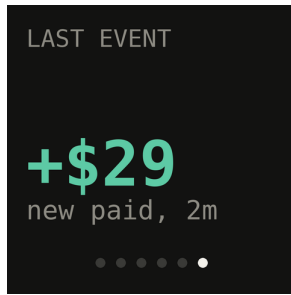
3. Paid subscribers. The count. Month-to-date change in the subtitle gives the number a trend without a chart.



4. Active trials. Not green: a trial is not yet revenue. The "ending soon" subtitle is the actionable part.



5. Trial conversion, 30 day. Consider omitting at low volume. A 34% figure computed over 11 trials swings wildly, and a jittery number undermines trust in the whole device.

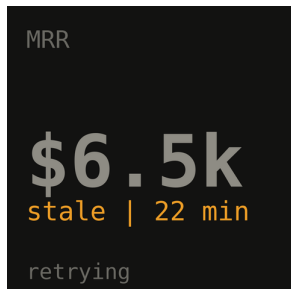


6. Latest event. The heartbeat. Confirms the device is live without a status indicator.

Conditional screen: add churn, but render it into rotation only when nonzero for the period, so it does not occupy 8 seconds displaying a permanent zero.

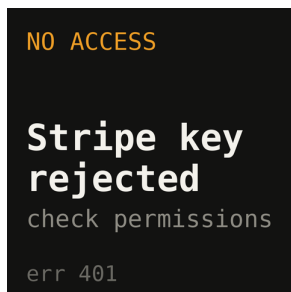
6.2 State screens

These determine whether the product feels like an instrument or a toy. Rotation screens are the easy path.



State A: stale. All values dim to muted, the age is shown in amber, and the retry status sits in the footer. This is the most important screen in the deck. A confidently displayed stale number is worse than an obviously stale one, and most cheap dashboards fail exactly here by freezing on a four-hour-old figure with no indication.

Trigger: last successful fetch older than 15 minutes.



State B: no access. Key revoked, wrong scope, or account access removed. Plain language for the user, error code for support. Must re-enter setup mode rather than silently showing stale data.



State C: setup. The only screen with 100% customer exposure, and where returns get decided. Deserves disproportionate polish. Shows the AP SSID and the next action. Firmware version in the footer for support.

7. Data layer

7.1 Endpoints

Three GET calls, no webhooks. Webhooks would require an inbound public endpoint, which is exactly the coupling this design avoids.

Purpose	Call	Payload size
MRR, paid count, trial count	GET /v1/subscriptions?status=active&limit=100&expand[]=data.discount	200-400 KB per page
Today's deltas, latest event	GET /v1/events?limit=20&created[gte]=<local_midnight_utc>	10-30 KB

Trial count is derived locally by filtering the subscriptions response on `status == "trialing"`, so it costs no additional request.

7.2 MRR computation

MRR is not a first-class Stripe field. Compute it:

```
mrr = 0
for sub in subscriptions where status in (active, trialing):
    for item in sub.items:
        price = item.price
        if price.recurring is null: continue # one-time, not MRR
        if price.billing_scheme == "tiered": flag() # cannot compute from price alone
        unit = price.unit_amount # cents
        qty = item.quantity
        n = price.recurring.interval_count
        switch price.recurring.interval:
        month: monthly = unit * qty / n
        year: monthly = unit * qty / (12 * n)
        week: monthly = unit * qty * 52 / (12 * n)
        day: monthly = unit * qty * 365 / (12 * n)
        mrr += monthly
    apply sub.discount # see below
```

Order of operations matters. Apply in this sequence:

1. **Discounts.** `sub.discount.coupon` gives `percent_off` or `amount_off`. Subtract before summing into the total. Skipping this makes a 50%-off annual plan read at double.
2. **Trials.** Count `status == "trialing"` separately. Do not add to MRR. That count is the TRIAL screen anyway.
3. **Tiered and metered prices.** `price.billing_scheme == "tiered"` has no `unit_amount` and cannot be computed from the price object alone. Irrelevant for flat-rate products; this is the line that breaks when usage pricing is added.

4. **Currency.** Sum only within a single currency. Mixed-currency accounts need a rate table the device will not have.

7.3 Polling strategy

Full recompute is expensive (200-400 KB). Use a hybrid:

- **Full recompute** every 10 minutes, on boot, and on any reconciliation trigger.
- **Incremental** every 60 seconds via `GET /v1/events?limit=10`. Adjust the cached MRR when `customer.subscription.created`, `.updated`, or `.deleted` appears. This also populates the latest-event screen for free.
- **Reconcile hourly** with a full recompute regardless, because incremental adjustment drifts.

7.4 Reliability requirements

A disconnected appliance has no fallback path. It must fail gracefully alone.

1. **Persist last-good state to flash.** On boot, render cached values immediately, then refresh. The screen is never blank.
2. **Show staleness, never a confident lie.** See State A.
3. **Exponential backoff on failure.** 60s, 120s, 240s, capped at 15 minutes. Stripe's read rate limit is far above anything this device generates, so backoff protects against cascade rather than throttling.
4. **NTP with explicit timezone offset.** `created[gte]` needs a correct epoch. Stripe returns UTC; local midnight is not UTC midnight. Without this, "today" resets at an arbitrary hour.
5. **Surface 401 visibly.** See State B.

7.5 The MRR definition problem

This will generate more support contact than anything technical. Users will compare the screen against their Stripe dashboard, ChartMogul, or Baremetrics and get a different number. Annual plans amortized monthly, trials counted or not, discounts applied or not, refunds, proration, and multi-currency all produce legitimate disagreement about what MRR "is."

Mitigation:

- Pick a definition and document it on one page.
- Expose two or three toggles in setup for the choices people care about most: include trials, annual amortization on or off.
- Ship the definition in the box.

8. Hardware

8.1 Display

ST7789 driver, 240x240 IPS, SPI interface. Standard for this form factor.

8.2 Compute options

	Raspberry Pi Zero 2 W	ESP32-S3
Effort	An evening, ~50 lines Python	A weekend or more
TLS	Handled by OS	mbedTLS, CA bundle in flash, rotation is your problem
JSON parsing	Buffer whole response in RAM	Must stream-parse; 300KB will not fit in heap
Boot time	20-30s	Under 2s
Power	~1.5W	~0.3W
Elegance	Lower	Higher

Recommendation for a solo builder: Pi Zero 2 W unless the embedded exercise is itself the goal. The ESP32 path is roughly 5x the hours for a better final object.

8.3 ESP32 streaming parse

If going the MCU route, you cannot buffer a 300 KB response in ~320 KB of usable heap. Use a filter document with ArduinoJson declaring only the needed paths:

- `data[].status`
- `data[].items.data[].quantity`
- `data[].items.data[].price.unit_amount`
- `data[].items.data[].price.recurring`
- `data[].items.data[].price.billing_scheme`
- `data[].discount`
- `has_more`

Everything else is discarded during parse. Accumulate the running total as bytes arrive.

9. Commercial considerations

9.1 Credential model

For a sold device, asking a customer to paste a Stripe API key into third-party hardware is a real trust ask, and a meaningful fraction will bounce. Stripe's own documentation discourages keys on end-user devices.

Path A: customer-provisioned restricted key (recommended to start)

Preserves the disconnected architecture, no backend required.

1. Device boots into AP mode, broadcasts Setup-XXXX, captive portal opens on the customer's phone.
2. WiFi credentials, stored in NVS.
3. Stripe restricted key field, with a deep link to the dashboard API keys page and explicit permission instructions: **Read** on Subscriptions, Events, Customers. **None** on everything else.
4. Validate immediately with GET `/v1/subscriptions?limit=1`. On success, show the customer their actual MRR on the phone before setup completes. That moment converts skeptics.

5. Preferences: timezone, currency, rotation contents, brightness schedule.

Worst case if the device is stolen: someone learns the owner's subscriber count. Compare that to a live secret key, where the worst case is issuing refunds.

To reduce drop-off at step 3:

- Ship a printed card with the exact permission list and a QR code to a one-page guide.
- Investigate Stripe's pre-filled restricted key creation URLs, which let you specify permissions in query parameters so the customer lands on a page with the right boxes ticked. Verify the current parameter format against Stripe's documentation before building against it, since this has changed over time.
- **Publish the firmware source.** "The key never leaves the device, and here is the code proving it" is far stronger than any assurance you could write, and costs nothing competitively. The moat is the hardware and the design, not 400 lines of polling logic.

Path B: Stripe Connect OAuth

What a corporate security team will expect. Customer authorizes via Stripe's own screen with `read_only` scope and never handles a key. Costs:

- You hold OAuth tokens for other people's Stripe accounts, a genuine security liability and eventually a SOC 2 conversation.
- Requires a Stripe Platform account and app review.
- Requires ongoing infrastructure. The device is no longer disconnected, and your uptime becomes their uptime.
- Devices become bricks if you shut down.

Start with Path A. Move to Path B only if selling into companies rather than to individual operators.

9.2 Brand and trademark

Do not use Stripe's wordmark, logo, or exact brand colors on the device or packaging. Use plain text ("works with Stripe") and include a "not affiliated with Stripe" line in the listing. This is a further argument for the neutral palette.

10. Build checklist

Design

- Every text element converted to millimeters and confirmed above 2.3mm
- Every string width-checked against the 208px column at its rendered size
- Hero size computed from string length, not hardcoded
- Palette round-tripped through RGB565 and visually confirmed on hardware
- Three-zone skeleton identical across all nine screens

Firmware

- Partial update window implemented (CASET / RASET), full redraw only on transitions
- Bitmap font set generated at final sizes

- Last-good state persisted to flash and rendered on boot
- Exponential backoff, capped at 15 minutes
- NTP sync with explicit local timezone offset
- Stale threshold at 15 minutes triggers State A
- 401 triggers State B and re-enters setup
- PWM night dimming schedule

Data

- Discounts applied before summing
- Trials excluded from MRR, counted separately
- Tiered pricing detected and flagged rather than silently miscomputed
- Single-currency assertion
- Hourly full reconciliation
- MRR definition documented, one page

Commercial

- Restricted key scoped to read-only on Subscriptions, Events, Customers
- Setup validates key and displays live MRR before completing
- Printed permission card in box
- Firmware source published
- "Not affiliated with Stripe" disclosure in listing
- Key rotation reminder documented for the customer

Appendix A: Quick reference constants

```
Panel diagonal 1.54 in
Panel side 27.7 mm
Resolution 240 x 240
Density 8.7 px/mm (~220 PPI)
Padding 16 px
Text column 208 px
Legibility floor 24 px (2.8 mm)
Absolute floor 20 px (2.3 mm)
Mono advance 0.6 em
Label baseline y = 16
Hero baseline y = 150
Subtitle baseline y = 178
Footer baseline y = 210
Rotation interval 8 s
Stale threshold 15 min
Full frame 115 KB (~25 ms @ 40 MHz SPI)
Partial frame 52.8 KB (~11 ms @ 40 MHz SPI)
Backlight life 20,000-30,000 hr
```

Appendix B: Sources

Stripe brand and design token values referenced in section 4.3:

- designsystems.one, Stripe Design System breakdown: <https://www.designsystems.one/design-systems/stripe-design>
- Brandfetch, Stripe brand assets: <https://brandfetch.com/stripe.com>
- Mobbin, Stripe brand color palette: <https://mobbin.com/colors/brand/stripe>

Stripe API behavior (subscription objects, price objects, restricted keys, Connect OAuth) should be verified against current documentation at <https://docs.stripe.com> before implementation, as these change.